

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Научно-учебная лаборатория
методов анализа больших
данных
Стажер-исследователь

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель
департамента программной
инженерии

_____ / М.В. Минец /
«__» _____ 2026 г.

_____ / Н.А. Павлочев /
«__» _____ 2026 г.

**СИСТЕМА РЕГИСТРАЦИИ И УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ
МЕРОПРИЯТИЙ (СЕРВЕРНАЯ ЧАСТЬ 1)**

Персональное техническое задание

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.01-01 ТЗ 01–2 ЛУ

Исполнители:
студент группы БПИ238

_____ / М.М. Апаркин /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.09.01-01 ТЗ 01–2 ЛУ

**СИСТЕМА РЕГИСТРАЦИИ И УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ
МЕРОПРИЯТИЙ (СЕРВЕРНАЯ ЧАСТЬ 1)**

Персональное техническое задание

RU.17701729.09.01-01 ТЗ 01–2

Листов 33

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	1
1.1. Наименование темы разработки	1
1.2. Наименование программы	1
1.3. Состав комплекса программ	1
1.4. Краткая характеристика области применения	2
2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ	3
2.1. Документы, на основании которых ведётся разработка	3
2.2. Назначение документа	3
2.3. Наименование темы разработки	4
2.4. Объём работ	4
3. НАЗНАЧЕНИЕ РАЗРАБОТКИ	5
3.1. Функциональное назначение программы	5
3.2. Эксплуатационное назначение программы	6
3.2.1. Архитектура развёртывания	6
3.2.2. Требования к окружению	7
3.2.3. Сценарии использования	8
3.2.4. Обслуживание и поддержка	9
4. ТРЕБОВАНИЯ К ПРОГРАММЕ ИЛИ ПРОГРАММНОМУ ИЗДЕЛИЮ	10
4.1. Требования к функциональным характеристикам	10
4.1.1. Управление темами (Management API)	10
4.1.2. Управление кластером (Management API)	10
4.1.3. Публикация сообщений (Data API)	11
4.1.4. Чтение сообщений (Data API)	11
4.1.5. Группы потребителей (Data API)	12
4.1.6. Алгоритм диапазонного назначения партиций	12
4.1.7. Аутентификация	12
4.2. Требования к надёжности	13
4.3. Требования к составу и параметрам технических средств	13
4.4. Требования к информационной и программной совместимости	14
4.5. Специальные требования	14
5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	15
5.1. Состав программной документации	15
5.2. Требования к документации источников кода и API	15
5.3. Специальные требования	16
6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	18
6.1. Экономическая эффективность	18
6.1.1. Снижение лицензионных затрат	18
6.1.2. Снижение требований к инфраструктуре	18

6.1.3. Прогнозируемые технические характеристики	18
7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ	20
7.1. Описание критических этапов	21
7.1.1. Этап 1.3: Проектирование инфраструктуры	21
7.1.2. Этап 2.4: Unit-тестирование	22
7.1.3. Этап 3.2: Нагрузочное тестирование	23
7.1.4. Этап 3.3: Подготовка документации	23
7.2. Критерии завершения каждого этапа	24
8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ	25
8.1. Критерии приёмки	25
8.2. Процедура приёмки	26
8.3. Требования к результатам приёмки	27
Events-service (HTTP API)	30
Payment-service	30
Qr-service	30

1. ВВЕДЕНИЕ

1.1. Наименование темы разработки

Наименование темы разработки — «**Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием**».

1.2. Наименование программы

Наименование программы — «**Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием**» (далее по тексту — «Брокер», «Система», «BunnyMQ»).

Наименование программы на английском языке — «**Go message queue with delivery guarantees and horizontal scaling**».

Краткое наименование: **BunnyMQ**.

Обозначение программного документа по ГОСТ 19.103-77: **RU.17701729.02-06 ТЗ 01–1** (числовой код вида документа по ГОСТ 19.101-77: **05**).

Вид программного изделия по ГОСТ 19.101-77, §2.2: **комплекс программ** — совокупность взаимодействующих программных компонентов, объединённых в единую систему: серверный процесс брокера, клиентская библиотека и утилита командной строки.

1.3. Состав комплекса программ

Комплекс BunnyMQ включает три исполняемых компонента и одну библиотеку:

1. `bunmq` (`cmd/bunmq`) — процесс брокера; запускает gRPC-серверы, узел Raft, координаторы и HTTP-эндпоинт метрик (Go 1.25);
2. `bunmq-cli` (`cmd/bunmq-cli`) — утилита командной строки для управления кластером и тестирования: команды `produce`, `consume`, `topic`, `cluster` (Go 1.25);
3. `pkg/client` — публичная Go-библиотека для встраивания в клиентский код: типы `Producer`, `Consumer`, `AdminClient` с автоматическим обнаружением лидера и повторными попытками (Go 1.25).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Серверные компоненты, используемые внутри брокера, не являются самостоятельными исполняемыми файлами, а входят в состав пакета `bunnymq`.

1.4. Краткая характеристика области применения

BunnyMQ предназначен для применения в инфраструктуре микросервисных приложений, где требуется надёжная асинхронная передача сообщений между компонентами системы с гарантиями доставки и возможностью горизонтального масштабирования.

Типовые сценарии использования:

- передача событий между микросервисами с гарантией «хотя бы однократной» доставки (`acks=all`);
- сбор данных телеметрии, логов или метрик от большого числа производителей с минимальными задержками (`acks=0`);
- организация нескольких независимых потоков данных в рамках одного кластера через механизм тем и партиций;
- параллельная обработка данных несколькими потребителями через группы потребителей с автоматическим перераспределением партиций.

Категории пользователей системы:

- **Разработчики приложений-производителей** — подключаются к кластеру BunnyMQ через клиентскую библиотеку или gRPC напрямую, отправляют сообщения в темы с выбором уровня подтверждения;
- **Разработчики приложений-потребителей** — читают сообщения из партиций начиная с заданного смещения, при необходимости координируют работу через группы потребителей;
- **Администраторы кластера** — управляют топологией кластера через `bunnymq-cli` или Management API: создают и удаляют темы, меняют настройки хранения, просматривают состояние кластера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

2.1. Документы, на основании которых ведётся разработка

Разработка серверной части 1 платформы CSTATI.events (events-service, payment-service, qr-service, инфраструктура) ведётся на основании следующих документов:

1. Устав федерального государственного автономного образовательного учреждения высшего образования «Национальный исследовательский университет «Высшая школа экономики»» (с изменениями) № 56 от 1.02.2016.
2. Приказ декана ФКН НИУ ВШЭ И. В. Аржанцева № 2.3-02/1112–04 от 11.12.2025 «Об утверждении тем, руководителей курсовых работ студентов образовательной программы Программная инженерия факультета компьютерных наук».
3. Техническое задание на платформу CSTATI.events в целом: «Система регистрации и управления жизненным циклом мероприятий» (RU.17701729.09.01-01 ТЗ 01–1).
4. Архитектурная документация системы CSTATI.events, определяющая взаимодействие микросервисов через REST API, gRPC и Apache Kafka.

2.2. Назначение документа

Настоящий документ является техническим заданием на разработку серверной части 1 (Серверная часть 1) платформы CSTATI.events, определяет требования к следующим компонентам:

1. **events-service** — микросервис управления жизненным циклом мероприятий, включая управление событиями, волнами регистрации, билетами, промокодами и операциями регистрации участников;
2. **payment-service** — микросервис обработки онлайн-платежей через интеграцию с платёжным шлюзом ЮKassa;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. **qr-service** — микросервис генерации и хранения QR-кодов электронных билетов;
4. **инфраструктура и DevOps** — Docker-контейнеризация, Docker Compose, Nginx (обратный прокси), PostgreSQL (базы данных для каждого сервиса), Apache Kafka (очереди сообщений), Redis (кэш), MinIO (хранилище объектов), мониторинг (Prometheus + Grafana), логирование и CI/CD-конвейер.

2.3. Наименование темы разработки

Наименование темы разработки — «Система регистрации и управления жизненным циклом мероприятий (Серверная часть 1)».

Расширенное наименование — «Платформа CSTATI.events: микросервис управления мероприятиями, интеграция платежей и QR-кодов».

2.4. Объём работ

Разработчик (М.М. Апаркин) отвечает за разработку, интеграцию и тестирование указанных выше трёх микросервисов, а также конфигурацию инфраструктуры и DevOps-конвейера. Остальные компоненты (auth-manager, notifier, tg-bot, frontend, Nginx) разрабатываются другими членами команды и интегрируются в единую систему в соответствии с определённым в общем ТЗ взаимодействием через REST API, gRPC и Kafka.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1. Функциональное назначение программы

Серверная часть 1 платформы CSTATI.events (events-service, payment-service, qr-service) реализует автоматизацию ключевых операций управления жизненным циклом мероприятий и обработки платежей:

1. **Управление мероприятиями** — создание, редактирование и удаление событий; управление статусами мероприятий; публикация мероприятий в открытом доступе; установка параметров события (дата, время, место, описание, ценовые категории).
2. **Управление волнами регистрации** — определение нескольких волн регистрации для одного события; автоматическое управление статусами волн (запланирована, активна, приостановлена, завершена); управление сроками открытия и закрытия волн.
3. **Управление билетными категориями** — определение нескольких типов билетов с разными ценами; настройка количества билетов в категории; атомарное резервирование мест; управление статусами билетов (активен, скрыт, распродан, архивирован).
4. **Управление промокодами** — создание кодов скидок с процентной скидкой; установка лимитов использования и сроков действия; атомарная активация промокодов с ограничением по частоте запросов (максимум 5 активаций/мин с одного IP).
5. **Регистрация участников** — приём заявок на участие с проверкой наличия свободных мест; применение промокодов и расчёт финальной стоимости; отправка заявки в очередь обработки Kafka; управление статусами регистрации (ожидание оплаты, подтверждена, отменена, истекла, неудачная, оплачена, пройден контроль).
6. **Обработка онлайн-платежей** — интеграция с платёжным шлюзом ЮKassa; создание платёжных ссылок с типом подтверждения redirect; обработка вебхуков о статусе платежей; обеспечение идемпотентности операций через

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ключи идемпотентности; поддержка платных и бесплатных билетов (для бесплатных обращение к ЮKassa не производится).

7. **Генерация QR-кодов** — генерирование PNG-изображений QR-кодов размером 256×256 пиксельной, кодирующих уникальный идентификатор регистрации; загрузка в объектное хранилище MinIO по протоколу S3; публикация публичных URL для доставки участникам.
8. **Контроль прохода на мероприятие** — верификация участников по QR-коду электронного билета; получение информации о регистрации на контрольном пункте; предотвращение повторного входа.
9. **Асинхронная обработка сообщений** — взаимодействие с Kafka для коммуникации с другими сервисами: публикация событий в топик `registrations`, `payments`, `qr-generation`; потребление сообщений из топика `ticket-status-updates` для обновления статуса билетов.

3.2. Эксплуатационное назначение программы

3.2.1. Архитектура развёртывания

Микросервисы `events-service`, `payment-service`, `qr-service` развёртываются в контейнеризированной среде Docker:

1. **Docker-контейнеризация**: каждый микросервис упаковывается в отдельный Docker-контейнер с указанием базового образа (`go:1.24-alpine`); управление контейнерами через Docker Compose в среде разработки;
2. **Постоянное хранилище**:
 - `events-service`: PostgreSQL 16 (БД `events_db`);
 - `payment-service`: PostgreSQL 16 (БД `payment_db`);
 - `qr-service`: PostgreSQL 16 (БД `qr_db`);
 - MinIO (S3-совместимое хранилище): QR-коды и прочие артефакты;
 - Redis 7: кэш Idempotency-Key для регистраций (TTL 24 ч).
3. **Очереди сообщений**: Apache Kafka для асинхронной коммуникации между `events-service`, `payment-service`, `notifier`, `tg-bot`:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- топик `registrations`: события о новых регистрациях;
- топик `payments`: события о статусах платежей;
- топик `qr-generation`: события о необходимости генерации QR-кода;
- топик `ticket-status-updates`: события о статусе билетов;
- топик `registrations-dlq`: Dead Letter Queue для неудачных сообщений.

4. **Сетевая архитектура:** все микросервисы находятся в одной Docker-сети; публичный доступ через Nginx (обратный прокси на порту 443 HTTPS) и gRPC-эндпоинты для внутренней коммуникации с auth-manager (порт 50051).

3.2.2. Требования к окружению

1. **Системные требования:** Linux-сервер (Ubuntu 20.04+) с минимум 4 ГБ оперативной памяти и 50 ГБ дискового пространства;
2. **Программное обеспечение:** Docker 20+, Docker Compose 2.0+;
3. **Внутренние зависимости:**
 - PostgreSQL 16 (три отдельные БД);
 - Apache Kafka 3.0+ с минимум 3 топиками;
 - Redis 7;
 - MinIO (для хранения QR-кодов);
 - auth-manager (gRPC на порту 50051) для проверки JWT-токенов;
 - notifier (Kafka-потребитель) для доставки уведомлений.
4. **Конфигурация окружения:** переменные окружения для каждого сервиса:
 - `DATABASE_URL` для подключения к PostgreSQL;
 - `KAFKA_BROKERS` для подключения к Kafka;
 - `REDIS_URL` для подключения к Redis;
 - `S3_ENDPOINT`, `S3_ACCESS_KEY`, `S3_SECRET_KEY` для MinIO;
 - `YUKASSA_API_KEY`, `YUKASSA_SHOP_ID` для платёжного шлюза ЮKassa;
 - `AUTH_SERVICE_GRPC_URL` для подключения к auth-manager.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2.3. Сценарии использования

Организатор (через REST API events-service):

- аутентифицируется с помощью JWT-токена от auth-manager;
- создаёт новое мероприятие, указывая название, описание, дату, место;
- настраивает волны регистрации и билетные категории;
- определяет промокоды с процентными скидками;
- просматривает список зарегистрировавшихся участников с фильтрацией по волне и статусу;
- экспортирует список гостей для печати.

Участник (через REST API events-service):

- просматривает доступные мероприятия и волны регистрации;
- регистрируется, заполняя форму (ФИО, email, Telegram);
- применяет промокод и видит финальную цену;
- переходит по платёжной ссылке, созданной payment-service;
- получает QR-код билета через Telegram от notifier;
- использует QR-код для входа на мероприятие.

Система (асинхронная обработка через Kafka):

- events-service публикует событие о новой регистрации в топик `registrations`;
- payment-service создаёт платёжную сессию и публикует о ней в топик `payments`;
- payment-service получает вебхук от ЮKassa и обновляет статус в БД и публикует в Kafka;
- qr-service потребляет события из топика `qr-generation`, генерирует QR-код и загружает в MinIO;
- события о завершении публикуются в топик `ticket-status-updates` для уведомления других сервисов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2.4. Обслуживание и поддержка

1. **Резервное копирование:** ежедневное резервное копирование БД PostgreSQL в архив (локальный диск или S3);
2. **Мониторинг:** сбор метрик через Prometheus с интервалом 30 секунд; визуализация в Grafana; оповещение об аномалиях;
3. **Логирование:** логи всех микросервисов сохраняются в папке `deploy/logs/` с ротацией по размеру (до 1 ГБ per file);
4. **Масштабируемость:** система рассчитана на 10–100 организаторов и 100–5 000 участников в зависимости от выделенных ресурсов контейнерам (CPU, память);
5. **Обновление версии:** миграция данных БД через `sqlc` и ручные скрипты миграции; откат возможен сохранением резервных копий перед обновлением.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММЕ ИЛИ ПРОГРАММНОМУ ИЗДЕЛИЮ

4.1. Требования к функциональным характеристикам

4.1.1. Управление темами (Management API)

Брокер должен реализовывать операции управления темами через Management API (gRPC-сервис ManagementService):

- Создание темы (CreateTopic):** входные параметры — name (уникальное имя, регулярное выражение $[a-zA-Z0-9._-]\{1,255\}$), partition_count (≥ 1), replication_factor ($\geq 1, \leq$ числа узлов кластера), опциональные параметры хранения (retention_ms, retention_bytes). При уже существующей теме возвращается код TopicAlreadyExists.
- Удаление темы (DeleteTopic):** физическое удаление данных выполняется асинхронно; при отсутствии темы возвращается TopicNotFound.
- Список тем (ListTopics):** возвращает список всех тем с полями name, partition_count, replication_factor.
- Описание темы (DescribeTopic):** возвращает полные метаданные темы, включая для каждой партиции: идентификатор лидера и список идентификаторов реплик с их статусом.
- Изменение числа партиций (AlterTopicPartitions):** разрешено только увеличение числа партиций; уменьшение не поддерживается.
- Изменение параметров хранения (AlterTopicRetention):** обновляет retention_ms и / или retention_bytes; новые параметры применяются на следующем цикле очистки.

4.1.2. Управление кластером (Management API)

- Описание кластера (DescribeCluster):** возвращает список узлов с полями node_id, address, status.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. **Список партиций темы** (`ListPartitions`): возвращает для каждой партиции `partition_id`, `leader_node_id`, `replica_node_ids`, диапазон смещений (`earliest_offset`, `latest_offset`).
3. Состав кластера задаётся статически в конфигурационном файле при запуске; добавление и удаление узлов в `runtime` в первой версии не поддерживается.

4.1.3. Публикация сообщений (Data API)

1. **Produce** (`Produce`): входные параметры — `topic`, `partition_id` (если не задан — выбирается по хешу ключа FNV-1a 32 или `round-robin`), `batch_data` (байтовая последовательность в формате Kafka-совместимого пакета), `acks` (0 или `all`). При `acks=all` возвращается назначенное смещение первого сообщения пакета; при `acks=0` возвращается ответ без смещения немедленно.
2. Все сообщения одного пакета записываются в одну партицию с последовательными смещениями атомарно (принцип «всё или ничего»).
3. Формат `batch_data` на диске совпадает с форматом на проводе; клиент кодирует пакет, сервер хранит его без изменений.

4.1.4. Чтение сообщений (Data API)

1. **Fetch** (`Fetch`): входные параметры — `topic`, `partition_id`, `offset`, `max_bytes`, `max_wait_ms`. Возвращает записи с последовательными смещениями начиная с `offset` и до исчерпания `max_bytes`. Если записей нет, запрос удерживается до появления данных или истечения `max_wait_ms`.
2. **GetOffsets** (`GetOffsets`): возвращает `earliest_offset` и `latest_offset` для заданной партиции; поддерживается запрос смещения по метке времени (`GetOffsetByTimestamp`).
3. Запросы `Fetch` и `GetOffsets` обслуживаются только на узле-лидере партиции; при обращении к не-лидеру возвращается `NotLeader` с адресом лидера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.1.5. Группы потребителей (Data API)

1. **JoinGroup**: потребитель указывает `group_id` и список тем. Сервер добавляет потребителя к группе, выполняет диапазонное назначение партиций, возвращает `member_id`, `generation_id` и список назначенных партиций.
2. **Heartbeat**: потребитель периодически (рекомендуется раз в 3 секунды) отправляет `heartbeat` с `group_id`, `member_id`, `generation_id`. Если `generation_id` устарел (произошла ребалансировка), ответ содержит сигнал `REBALANCE_REQUIRED` — потребитель должен повторить `JoinGroup`.
3. **LeaveGroup**: потребитель явно покидает группу; происходит немедленная ребалансировка с новым `generation_id`.
4. **CommitOffset**: потребитель фиксирует смещения по одной или нескольким партициям. Для фиксации смещений требуется действующий `member_id` и актуальный `generation_id`; при несоответствии возвращается ошибка.
5. **FetchCommittedOffsets**: возвращает последние зафиксированные смещения для заданных пар (`group_id`, `partition`).
6. Если потребитель не отправлял `heartbeat` дольше `session timeout` (по умолчанию 30 секунд), координатор исключает его из группы и инициирует ребалансировку.

4.1.6. Алгоритм диапазонного назначения партиций

Для каждой темы: партиции сортируются по `partition_id`, члены группы сортируются по `member_id`. Диапазон `partition_ids` делится на `len(members)` равных частей; остаток (если число партиций не кратно числу членов) прибавляется к первым членам. Результат — список (`member_id`, `[partition_ids]`).

4.1.7. Аутентификация

Клиент передаёт токен в заголовке gRPC-метаданных `bunnumq-auth-token`. Сервер проверяет токен по списку, заданному в конфигурации при запуске. Если список пуст — аутентификация отключена (режим `PLAINTEXT`). Недействительный токен возвращает `Unauthenticated`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.2. Требования к надёжности

1. При отказе одного узла из трёх кластер продолжает обслуживать запросы (кворум Raft).
2. Данные, подтверждённые с `acks=all`, не теряются при отказе одного узла.
3. Узел после перезапуска восстанавливает состояние Metadata FSM из snapshot и replaying команд Raft; Partition FSM восстанавливает Storage из журнала на диске.
4. Storage корректно обрабатывает ситуацию torn write (незавершённая запись последнего пакета): при открытии активного сегмента хвост усекается до последней полной записи.
5. Очистка сегментов по политике хранения (`DeleteSegmentsBefore`) реализована как команда Raft, что гарантирует одинаковое удаление данных на всех репликах.
6. Операция graceful shutdown: узел прекращает приём новых RPC, ожидает завершения текущих, синхронизирует Storage и корректно завершает процесс dragonboat.

4.3. Требования к составу и параметрам технических средств

Таблица 4.1 — Минимальные требования к аппаратным средствам

Параметр	Минимальное значение
CPU	2 ядра x86-64 на узел
Оперативная память	512 МБ на узел (без учёта данных)
Дисковое хранилище	SSD или HDD; объём определяется <code>retention_bytes</code> × число партиций
Операционная система	Linux (ядро 5.4+) или macOS 12+
Сеть	TCP-связность между всеми узлами; задержка < 100 мс

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.4. Требования к информационной и программной совместимости

1. Язык реализации: Go 1.25 и выше.
2. Внешний транспорт: gRPC (HTTP/2), Protobuf. Совместимость на уровне бинарного протокола с любым gRPC-клиентом, использующим сгенерированные из `api/*.proto` stub-файлы.
3. Протокол Raft: библиотека dragonboat v4 (github.com/ltni/dragonboat/v4); взаимодействие между узлами только через dragonboat, сторонние реализации Raft несовместимы.
4. Метрики: формат Prometheus text/plain, совместимый с Prometheus Server 2.x и OpenMetrics.
5. Формат журнала (log): хранение совместимо с форматом пакетов Apache Kafka; это позволяет в будущем переключить транспорт без изменения слоя хранения.
6. Зависимостей на внешние базы данных, очереди или объектные хранилища нет; всё хранится локально на диске каждого узла.

4.5. Специальные требования

1. Partition FSM (IOndiskStateMachine) строго детерминирована: метод `Update()` не вызывает `time.Now()`, системных вызовов без гарантированного результата и сетевых операций.
2. Логирование: структурированный JSON через go.uber.org/zap; уровни `debug`, `info`, `warn`, `error`.
3. Отсутствие глобального состояния между запросами; конкурентность управляется через `goroutine` и `channel`.
4. Метрики экспортируются на HTTP-эндпоинте `/metrics`; порт настраивается в конфигурации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1. Состав программной документации

Разработка микросервисов `events-service`, `payment-service` и `qr-service` должна сопровождаться следующей программной документацией в соответствии с ГОСТ 19.201–78:

Командная документация (разрабатывается совместно):

1. **Техническое задание на серверную часть 1** (ГОСТ 19.201–78) — настоящий документ.
2. **Программа и методика испытаний** (ГОСТ 19.301–79) — документ, описывающий методы проверки соответствия разработанного ПО требованиям ТЗ.

Индивидуальная документация (разрабатывается Апаркиным):

3. **Текст программы** (ГОСТ 19.401–78) — исходный код микросервисов (Go 1.24), форматированный согласно `gofmt` и `golangci-lint`.
4. **Пояснительная записка** (ГОСТ 19.404–79) — описание архитектуры `events-service`, `payment-service`, `qr-service`; дизайн-решения; обоснование выбора технологий; анализ сложности алгоритмов; описание тестирования.
5. **Руководство оператора** (ГОСТ 19.505–79) — инструкции по развёртыванию сервисов в `Docker`; конфигурация; управление; мониторинг; обслуживание.

5.2. Требования к документации источников кода и API

API-документация:

- Все REST API-эндпоинты должны быть задокументированы в формате OpenAPI 3.0 (YAML);
- Описание параметров запроса, ответа, кодов ошибок для каждого эндпоинта;
- Примеры запросов и ответов для основных операций;
- Документированы все Kafka-топики: имя, структура сообщений (JSON), частота публикации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Документация БД:

- Описание схемы PostgreSQL для `events_db`, `payment_db`, `qr_db` с перечислением таблиц, колонок, индексов, ограничений;
- Описание ER-диаграмм (Entity-Relationship Diagrams) для наглядности;
- Документирование ключевых хранимых процедур (если используются).

Документация кода:

- Inline-комментарии в коде на русском или английском языке;
- Документированные функции (GoDoc для Go) с описанием параметров и возвращаемых значений;
- Примеры использования сложных функций;
- Файл `README.md` для каждого микросервиса с описанием назначения, зависимостей, инструкциями по сборке и запуску.

Документация инфраструктуры:

- Описание `docker-compose.yml` и переменных окружения;
- Описание `Dockerfile` для каждого микросервиса;
- Инструкции по настройке Kafka-топиков и потребителей;
- Инструкции по миграции БД и инициализации схемы.

Документация тестирования:

- Описание unit-тестов и integration-тестов;
- Инструкции по запуску тестов (`go test ./...`);
- Отчёт о покрытии кода (coverage report);
- Описание сценариев нагрузочного тестирования.

5.3. Специальные требования

1. Все документы оформляются в соответствии с ГОСТ 19.106–78 (размер листа А4, поля, шрифт, нумерация);
2. Пояснительная записка загружается в систему Антиплагиат НИУ ВШЭ и представляется справка о загрузке;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. Все исходные коды размещаются в репозитории Git (GitHub/GitLab) с правильной структурой папок и история коммитов;
4. Электронные копии всей документации (ТЗ, ПМИ, ТП, ПЗ, РО) и исходного кода упаковываются в архив .zip или .rar с чёткой структурой папок.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1. Экономическая эффективность

Разработка выполняется в рамках учебного курсового проекта; коммерческая оценка стоимости не является целью ТЗ. Тем не менее использование BunnyMQ как решения с открытым исходным кодом имеет измеримые преимущества по сравнению с коммерческими альтернативами.

6.1.1. Снижение лицензионных затрат

Существующие промышленные брокеры сообщений с сопоставимыми гарантиями доставки (Apache Kafka в managed-варианте, Amazon SQS / Amazon MSK, Confluent Cloud) имеют стоимость эксплуатации от нескольких сотен до десятков тысяч долларов в месяц в зависимости от объёма трафика. BunnyMQ распространяется с открытым исходным кодом, что позволяет развернуть его на собственной инфраструктуре без лицензионных отчислений.

6.1.2. Снижение требований к инфраструктуре

Зависимости BunnyMQ ограничиваются одним исполняемым файлом на узел и дисковым хранилищем. Отсутствует необходимость в отдельном ZooKeeper-кластере (в отличие от Apache Kafka версий до 3.x), внешней базе данных или объектном хранилище. Это снижает операционную сложность и стоимость инфраструктуры.

6.1.3. Прогнозируемые технические характеристики

Таблица 6.1 — Ожидаемые технические характеристики BunnyMQ

Характеристика	Ожидаемое значение
Пропускная способность (acks=0, последовательная запись)	$\geq 100\,000$ сообщ./с на узел (размер 1 КБ)
Пропускная способность (acks=all, RF=3)	$\geq 10\,000$ сообщ./с на кластер
Задержка записи (p99, acks=all, RF=3, LAN)	≤ 50 мс
Задержка чтения (cached, long-poll timeout 0)	≤ 5 мс
Коэффициент готовности кластера (3 узла)	$\geq 0,999$ (переживает отказ одного узла)
Максимальный объём хранимых данных на узел	ограничен только ёмкостью диска

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Показатели производительности определены расчётным методом на основе бенчмарков последовательной записи на SSD (результаты в `benchmarks/disk_test.go`) и характеристик библиотеки dragonboat v4.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Разработка серверной части 1 платформы CSTATI.events (events-service, payment-service, qr-service, инфраструктура и DevOps) выполняется в три основные стадии в соответствии с ГОСТ 19.102–77.

Таблица 7.1 — Стадии и этапы разработки для Серверной части 1

Стадия разработки	Этап работ	Содержание работ	Завершение
1. Техническое проектирование	Анализ требований	Изучение ТЗ, анализ взаимодействия с другими микросервисами (auth-manager, notifier, tg-bot), определение границ ответственности events-service, payment-service, qr-service	15.01.26
	Проектирование архитектуры	Проектирование ER-диаграмм для каждой БД; проектирование REST API (OpenAPI); определение Kafka-топиков и сообщений; выбор фреймворков (Echo v4, sqlc, oapi-codegen)	15.01.26
	Проектирование инфраструктуры	Определение Docker-контейнеров для каждого сервиса; определение внешних зависимостей (PostgreSQL, Kafka, Redis, MinIO); подготовка docker-compose.yml и переменных окружения; настройка nginx для маршрутизации	15.01.26
2. Рабочее проектирование и разработка	Реализация events-service	Реализация CRUD-операций для мероприятий, волн, билетов, промокодов; реализация регистрации участников; реализация обработчиков Kafka (RegistrationWorker, PaymentWorker, ExpirationWorker); валидация входных данных; обработка ошибок	01.03.26
	Реализация payment-service	Реализация интеграции с ЮKassa; создание платёжных ссылок; обработка вебхуков; обеспечение идиоматичности; логирование операций	01.03.26

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Стадия разработки	Этап работ	Содержание работ	Завершение
3. Интеграция и испытания	Реализация qr-service	Реализация потребителя Kafka топика qr-generation; реализация генерации PNG-QR-кодов; загрузка в MinIO; публикация статуса в Kafka	01.03.26
	Unit-тестирование	Написание unit-тестов для всех компонентов с целью достичь покрытия кода $\geq 80\%$; тестирование граничных случаев; тестирование обработки ошибок	10.03.26
	Интеграционное тестирование	Интеграция с PostgreSQL, Kafka, Redis, MinIO; проверка сквозного сценария регистрации; проверка взаимодействия с auth-manager (gRPC); проверка взаимодействия с notifier и tg-bot через Kafka	20.03.26
	Нагрузочное и надёжностное тестирование	Тестирование производительности (p99 latency ≤ 500 мс, пропускная способность ≥ 100 регистраций/сек); тестирование отказоустойчивости при отключении зависимостей; проверка механизма DLQ	25.03.26
	Подготовка документации и деплой	Написание пояснительной записки; подготовка руководства оператора; подготовка документации API; инструкции по развёртыванию; загрузка в Антиплагиат; финальная сборка и деплой на staging-окружение	01.04.26

7.1. Описание критических этапов

7.1.1. Этап 1.3: Проектирование инфраструктуры

На этом этапе определяется полная архитектура развёртывания:

- Структура docker-compose.yml с сервисами: events-service, payment-service, qr-service, postgres (три отдельные БД), kafka, kafka-ui, redis, minio, prometheus, grafana;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- Определение переменных окружения (`.env` файл) для каждого окружения (dev, staging, production);
- Конфигурация Nginx для маршрутизации запросов к events-service, payment-service;
- Настройка Kafka-топиков с необходимыми параметрами (partitions, replication factor, retention).

7.1.2. Этап 2.4: Unit-тестирование

Обязательно тестируются следующие компоненты:

events-service:

- создание/редактирование/удаление мероприятия;
- переходы статусов волн (scheduled → active → closed);
- атомарное резервирование мест с оптимистичной блокировкой;
- применение промокодов и расчёт скидок;
- обработчики Kafka (RegistrationWorker, PaymentWorker, ExpirationWorker);
- обработка граничных случаев (все места забронированы, истёк срок регистрации, некорректный промокод).

payment-service:

- создание платёжной сессии в ЮKassa;
- обработка вебхука об успешной/неудачной оплате;
- идемпотентность (повторная обработка одного вебхука не создаёт дублирующие транзакции);
- обработка платных и бесплатных билетов.

qr-service:

- генерирование PNG-QR-кода из идентификатора регистрации;
- загрузка в MinIO и получение публичного URL;
- обработка события из Kafka топика qr-generation.

Все тесты запускаются командой `go test ./... -cover` и должны выдавать результат PASS.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

7.1.3. Этап 3.2: Нагрузочное тестирование

Система должна выдержать следующую нагрузку:

- **Регистрации:** минимум 100 регистраций/сек (при условии достаточных ресурсов контейнеров);
- **Платежи:** минимум 50 платежей/сек обрабатываются асинхронно через Kafka;
- **Kafka throughput:** $\geq 10\,000$ сообщений/мин по всем топикам;
- **Одновременные подключения:** система обслуживает до 1 000 одновременных HTTP-соединений без деградации.

Для тестирования используется инструмент k6 или локальный скрипт на Go.

7.1.4. Этап 3.3: Подготовка документации

Пояснительная записка должна содержать:

- описание архитектуры и взаимодействия трёх микросервисов;
- описание схем БД (ER-диаграммы);
- описание REST API с примерами;
- описание Kafka-топиков и обработчиков;
- описание алгоритмов критических операций (резервирование, идемпотентность);
- результаты тестирования (покрытие кода, результаты нагрузочного теста);
- инструкции по развёртыванию и запуску.

Руководство оператора должно содержать:

- системные требования;
- инструкции по установке Docker и Docker Compose;
- инструкции по конфигурации переменных окружения;
- инструкции по запуску, остановке, перезагрузке сервисов;
- инструкции по просмотру логов;
- инструкции по резервному копированию БД;
- инструкции по обновлению версии ПО.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

7.2. Критерии завершения каждого этапа

1. Техническое проектирование завершено, когда:

- все ER-диаграммы согласованы с архитектурой системы;
- OpenAPI-схема REST API согласована с другими командами;
- структура docker-compose.yml определена и одобрена;
- план тестирования утвержден.

2. Рабочее проектирование завершено, когда:

- реализованы все функции, перечисленные в разделе 4 ТЗ;
- все unit-тесты проходят, покрытие кода $\geq 80\%$;
- код прошёл проверку статическими анализаторами (golangci-lint);
- код отформатирован в соответствии со стандартами (gofmt).

3. Интеграция завершена, когда:

- все микросервисы успешно взаимодействуют через API, Kafka;
- интеграционные тесты проходят полностью;
- система корректно обрабатывает полный цикл регистрации и оплаты.

4. Испытания завершены, когда:

- результаты нагрузочного тестирования соответствуют требованиям раздела 4;
- система восстанавливается после сбоев без потери данных;
- составлен и подписан акт приёмки.

5. Документация завершена, когда:

- пояснительная записка загружена в Антиплагиат;
- справка о загрузке получена;
- все документы упакованы в архив и готовы к сдаче.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ

8.1. Критерии приёмки

BunnyMQ считается готовым к приёмке при выполнении всех следующих условий:

1. Критерии качества кода:

- `go build ./...` завершается без ошибок;
- `go test ./...` проходит без ошибок на всех пакетах;
- `golangci-lint run ./...` не выдаёт критических предупреждений;
- покрытие кода тестами для пакетов `internal/storage` и `internal/metadata` не менее 80 %.

2. Критерии функциональности:

- все gRPC-методы Data API и Management API, перечисленные в разделе 4 ТЗ, реализованы и принимают запросы;
- кластер из 3 узлов корректно создаёт тему, принимает сообщения с `acks=all` и отдаёт их через `Fetch`;
- при аварийной остановке одного узла из трёх кластер продолжает обслуживать запросы;
- Metadata FSM корректно восстанавливается после перезапуска узла из `snapshot`;
- Storage корректно выполняет `recover`y после незавершённой записи (`torn write`).

3. Критерии надёжности:

- сообщения, подтверждённые с `acks=all`, не теряются при отказе одного узла;
- очистка по политике хранения (`DeleteSegmentsBefore`) применяется детерминированно на всех репликах через команду `Raft`.

4. Критерии производительности:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- пропускная способность последовательной записи на диск при размере блока 4 096 байт и без `O_SYNC` ≥ 500 МБ/с на тестовом стенде (подтверждается `benchmarks/disk_test.go`).

5. Критерии групп потребителей:

- группа из двух потребителей корректно делит партии темы через `JoinGroup`;
- при выходе одного потребителя (`LeaveGroup` или `session timeout`) производится ребалансировка без потери данных;
- зафиксированные смещения сохраняются в `Metadata FSM` и доступны после перезапуска потребителя.

8.2. Процедура приёмки

1. **Функциональное тестирование** — проверка каждого gRPC-метода через `bunnumq-cli` или автоматические тесты по сценариям из Программы и методики испытаний.
2. **Кластерное тестирование** — запуск трёх процессов `bunnumq` на локальной машине (разные порты); проверка `produce/fetch`, `failover` при убийстве одного узла, восстановления после перезапуска.
3. **Модульное тестирование** — запуск `go test ./...`; проверка результатов покрытия.
4. **Тестирование Storage** — проверка корректности записи и чтения, сегментного перехода, индексного поиска, восстановления после `torn write`.
5. **Бенчмарки** — запуск `go test ./benchmarks/... -bench=. -benchmem`; проверка соответствия характеристикам из таблицы 6.1.
6. **Проверка документации** — наличие всех документов состава, перечисленных в разделе 5 ТЗ.
7. **Составление акта приёмки** — по результатам составляется акт, подписываемый исполнителем и руководителем.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

8.3. Требования к результатам приёмки

- `go test ./...` — статус PASS для всех пакетов.
- Отсутствие data-race при запуске `go test -race ./...`
- Покрытие `internal/storage` не менее 80 % по отчёту `go tool cover`.
- Кластер из 3 узлов успешно завершает сквозной тест: `CreateTopic` → `Produce(acks=all) × 100` → `Fetch` всех сообщений → проверка смещений.
- Отсутствие потери подтверждённых сообщений после `kill` одного узла и переизбрания лидера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ГЛОССАРИЙ И СОКРАЩЕНИЯ

Термины и сокращения, используемые в настоящем документе:

API — Application Programming Interface — программный интерфейс взаимодействия.

CI/CD — Continuous Integration / Continuous Deployment — практика непрерывной интеграции и доставки программного обеспечения.

DLQ — Dead Letter Queue — очередь недоставленных сообщений.

Kafka — Apache Kafka — брокер событий для асинхронного обмена между сервисами.

MinIO/S3 — S3-совместимое объектное хранилище для QR-артефактов.

PostgreSQL — Реляционная СУБД, используемая для транзакционной бизнес-логики сервисов.

Redis — In-memory хранилище для идемпотентности, блокировок и кэша.

QR-код — Двумерный код электронного билета для check-in участника.

ЮKassa — Платежный шлюз для онлайн-оплаты билетов.

Nginx — Обратный прокси и точка входа внешнего периметра.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПЕРЕЧЕНЬ КАФКА-ТОПИКОВ

Таблица 2.1 — Kafka-топики в зоне ответственности исполнителя

Топик	Парт.	Производитель	Потребитель
registrations	3	events-api	events-worker
payments	3	events-worker, payment-service	payment-service, events-worker
notifications	3	events-worker, payment-service, qr-service	integration, потребители, уведомления
qr-generation	3	events-worker	qr-service
registrations-dlq	1	events-worker	ручные обработки, алерты

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПЕРЕЧЕНЬ ОСНОВНЫХ API-ЭНДПОИНТОВ

Events-service (HTTP API)

Таблица 3.1 — Основные эндпоинты events-service

Метод	Путь	Назначение
GET	/api/v1/events	Список мероприятий
POST	/api/v1/events	Создание мероприятия
PATCH	/api/v1/events/{event_id}	Изменение мероприятия
POST	/api/v1/registrations	Создание регистрации
GET	/api/v1/events/{event_id}/guests	Список гостей
POST	/api/v1/checkin/{registration_id}	Подтверждение check-in

Payment-service

Таблица 3.2 — Основные эндпоинты payment-service

Метод	Путь	Назначение
POST	/api/v1/payment/webhook	Webhook статуса платежа
GET	/api/v1/payment/success	Возврат пользователя после оплаты
GET	/api/v1/payment/health	Проверка работоспособности

Qr-service

Таблица 3.3 — Основные эндпоинты qr-service

Метод	Путь	Назначение
GET	/api/qr/health	Проверка работоспособности

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 4**СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ**

1. ГОСТ 19.101–77 Виды программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102–77 Стадии разработки // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103–77 Обозначения программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104–78 Основные надписи // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105–78 Общие требования к программным документам // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106–78 Требования к программным документам, выполненным печатным способом // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201–78 Техническое задание. Требования к содержанию и оформлению // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301–79 Программа и методика испытаний // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401–78 Текст программы // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404–79 Пояснительная записка // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505–79 Руководство оператора // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603–78 Общие правила внесения изменений // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
13. The Go Programming Language Specification. Версия 1.25. [Электронный ресурс]. — URL: <https://go.dev/ref/spec> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

14. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm (Extended Version). USENIX ATC „14, 2014. [Электронный ресурс]. — URL: <https://raft.github.io/raft.pdf> (дата обращения: 26.04.2026).
15. dragonboat: A Multi-Group Raft library in Go. Документация dragonboat v4. [Электронный ресурс]. — URL: <https://github.com/lni/dragonboat> (дата обращения: 26.04.2026).
16. Apache Kafka Documentation. Версия 3.7. [Электронный ресурс]. — URL: <https://kafka.apache.org/documentation/> (дата обращения: 26.04.2026).
17. gRPC Documentation. [Электронный ресурс]. — URL: <https://grpc.io/docs/> (дата обращения: 26.04.2026).
18. Protocol Buffers Documentation. Версия proto3. [Электронный ресурс]. — URL: <https://protobuf.dev/> (дата обращения: 26.04.2026).
19. Kreps J., Narkhede N., Rao J. Kafka: a Distributed Messaging System for Log Processing. NetDB „11, 2011. [Электронный ресурс]. — URL: <https://notes.stephenholiday.com/Kafka.pdf> (дата обращения: 26.04.2026).
20. Uber go/zap. Blazing fast, structured, leveled logging in Go. [Электронный ресурс]. — URL: <https://github.com/uber-go/zap> (дата обращения: 26.04.2026).
21. Prometheus Documentation. [Электронный ресурс]. — URL: <https://prometheus.io/docs/> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]